

movaddsubshift.S

```

1  @ =====
2  @ 示範：MOV、SUB、Shift 指令的 64 位元減法運算
3  @
4  @ 學號：U13157005
5  @ 姓名：吳昊翊弘
6  @
7  @ =====
8
9  .thumb_func          @ 宣告此函式使用 Thumb 指令集（為配合 SDK 中的 BLX 跳躍指令所需）
10
11 .global main          @ 提供程式的起始進入點位址給連結器（Linker）
12
13 main:
14     BL      stdio_init_all
15 @ 初始化 Raspberry Pi Pico 的標準輸出（預設啟用 USB 序列埠）
16
17 after:
18     MOV     R2, #3      @ 將立即值 3 載入暫存器 R2 (R2 = 3)
19     MOV     R1, R2      @ 將 R2 的值複製到 R1，此時 R1 也是 3
20     MOVS    R1, R2      @ 加上 S (MOVS) 代表複製數值，並根據結果更新 CPU 的狀態旗標
21     (如 Z, N 旗標)
22     MOV     R9, R2      @ 將 R2 的值複製到高位暫存器 R9 (R9 = 3)
23
24 @ 在完全了解暫存器之前，不應隨意更改 SP（堆疊指標）或 PC（程式計數器）
25
26     ADD     R4, R2, #7   @ R4 = R2 + 7。這種格式的立即值限制為 3 位元（範圍 0~7）
27     @ 此時 R4 = 3 + 7 = 10
28
29     ADD     R4, R4, #255 @ R4 = R4 + 255。此格式允許 8 位元立即值（範圍 0~255）
30     @ 此時 R4 = 10 + 255 = 265
31
32     ADD     R4, #255     @ 簡寫形式，等同於 R4 = R4 + 255
33     @ 此時 R4 = 265 + 255 = 520
34
35     MOV     R7, #23      @ 先將 23 載入低位暫存器 R7
36     MOV     R11, R7      @ 接著將值搬移到高位暫存器 R11 (R11 = 23)
37
38     MOV     R7, #54      @ 先將 54 載入低位暫存器 R7
39     MOV     R10, R7      @ 接著將值搬移到高位暫存器 R10 (R10 = 54)
40
41     ADD     R10, R10, R11 @ 計算 R10 = R10 + R11（少數允許高位暫存器直接相加的指令）
42     @ 此時 R10 = 54 + 23 = 77
43
44     ADD     SP, SP, #508 @ 移動堆疊指標 SP（通常不應在沒有對應 SUB 的情況下直接操作）
45     SUB     SP, SP, #508 @ 將 SP 還原，復原剛才對堆疊造成的變動
46
47     ADD     R4, SP, #1020 @ 以 SP 為基準加入 8 位元倍數為 4 的立即值（0~1020），結果會
48     依 SP 位址而變動
49
50 @-----
51 @ 位移與循環移位指令示範 (Shift instructions)
52 @-----

```

```

50
51     MOV     R3, #8           @ R3 = 8 (稍後用作位移量，代表 1 位元組/8位元)
52     MOV     R2, #0xFF       @ R2 = 0xFF (十進位為 255)
53     MOV     R1, #4           @ R1 = 4
54
55     LSL     R1, R1, #2       @ 邏輯左移 (Logical Shift Left)：將 R1 左移 2 位元 (等同乘
以 4)
56
57
58     LSL     R1, #2           @ 簡寫形式 (省略目的地)：將 R1 左移 2 位元
59     @ R1 = 4 << 2 = 16
60
61     LSR     R1, R2, #8       @ 邏輯右移 (Logical Shift Right)：將 R2 右移 8 位元並存入
R1
62     @ R1 = 0xFF >> 8 = 0
63
64     LSR     R1, R3           @ 將 R1 右移，位移量為 R3 暫存器中的值 (即右移 8 位元)
65     ASR     R1, #8           @ 算術右移 (Arithmetic Shift Right)：將 R1 右移 8 位元，但
保留原本的最高位元正負號
66     ROR     R1, R3           @ 循環右移 (Rotate Right)：將 R1 的位元往右邊循環滾動 8 位
元
67
68 @-----
69 @ 範例：如何將 32 位元數值 0x12345678 分次拼接到 R3 中
70 @ (因為 Thumb 16位元指令限制，MOV 無法一次載入大於 8 位元的立即值)
71 @-----
72
73     MOV     R3, #0x12       @ 載入最高位元組 (R3 = 0x00000012)
74     LSL     R3, #24         @ 左移 24 位元 (R3 = 0x12000000)
75
76     MOV     R1, #0x34       @ 載入次高位元組 (R1 = 0x00000034)
77     LSL     R1, #16         @ 左移 16 位元 (R1 = 0x00340000)
78     ADD     R3, R1          @ 累加到 R3 (R3 = 0x12340000)
79
80     MOV     R1, #0x56       @ 載入次低位元組 (R1 = 0x00000056)
81     LSL     R1, #8          @ 左移 8 位元 (R1 = 0x00005600)
82     ADD     R3, R1          @ 累加到 R3 (R3 = 0x12345600)
83
84     MOV     R1, #0x78       @ 載入最低位元組 (R1 = 0x00000078)
85     ADD     R3, R1          @ 最後相加，完成拼接 (R3 = 0x12345678)
86

```

```

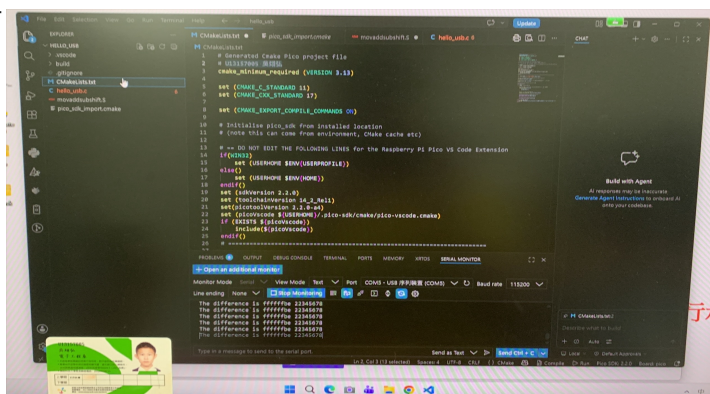
87 @-----
88 @ 準備 64 位元減法暫存器
89 @
90 @ 第一個數 (被減數)：
91 @     高位 R2 | 低位 R3
92 @     0x12   | 0x12345678
93 @
94 @ 第二個數 (減數)：
95 @     高位 R4 | 低位 R5
96 @     0x54   | 0xF0000000
97 @-----

```

```

98
99     MOV     R2, #0x12       @ 設定第一個數的高位字組 (0x00000012)

```



```

100      MOV      R4, #0x54          @ 設定第二個數的高位字組 (0x00000054)
101
102      MOV      R5, #0xf0          @ 將 0xF0 載入低位暫存器 R5
103      LSL      R5, #24            @ 左移 24 位元，使 R5 = 0xF0000000 (第二個數的低位字組)
104
105 @-----
106 @ 64 位元減法 (64-bit Subtraction)
107 @
108 @ 被減數: 0x00000012_12345678
109 @ 減數 : - 0x00000054_F0000000
110 @ -----
111 @ 低位運算 (R3 - R5) :
112 @ 0x12345678 < 0xF0000000 (發生不夠減，會向高位借 1)
113 @ 低位計算結果 = 0x12345678 - 0xF0000000 = 0x22345678
114 @
115 @ 高位運算 (R2 - R4) :
116 @ 原本為 0x12 - 0x54 = -0x42 (0xFFFFFBE)
117 @ 【提示】若此處使用帶借位減法指令 (如 SBCS)，會再額外扣掉低位產生的借位 1
118 @-----
119
120      SUB      R3, R5              @ 低位字組相減：R3 = R3 - R5 (0x12345678 - 0xF0000000)
121      SUB      R2, R4              @ 高位字組相減：R2 = R2 - R4 (0x12 - 0x54)
122
123      MOV      R0, R2              @ 將計算後的高位結果存入 R0，作為回傳值/輸出的高位參數
124      MOV      R1, R3              @ 將計算後的低位結果存入 R1，作為回傳值/輸出的低位參數
125
126 @ 備份結果 R0、R1 到 R6、R7 (因為稍後呼叫的 C 語言 printf 函式會覆寫 R0-R3 暫存器)
127
128      MOV      R6, R0              @ R6 = 高位差值
129      MOV      R7, R1              @ R7 = 低位差值
130
131 @ -----
132 @ 循環輸出結果 (Print result)
133 @ -----
134 loop:
135      MOV      R1, R6              @ 將備份的高位差值放入 R1，作為 printf 的第二個參數 (對應第一
個 %x)
136      MOV      R2, R7              @ 將備份的低位差值放入 R2，作為 printf 的第三個參數 (對應第二
個 %x)
137      LDR      R0, =sumstr          @ 將格式化字串 sumstr 的記憶體位址載入 R0，作為 printf 的第
一個參數
138      BL       printf              @ 呼叫 C 語言 stdio 的 printf 函式，經由 USB 序列埠輸出文字
139      B        loop                @ 無條件跳躍回 loop 標籤，形成無窮迴圈
140
141 .data
142 .align 4                          @ 進行 4 位元組記憶體對齊，加速硬體讀取
143
144 sumstr:
145 .asciz "The difference is %x %x\n" @ 定義以零結尾 (\0) 的 C 語言字串，顯示差值結果

```